

一、操作系统

Q1: 进程与线程的区别是什么?

(1) **资源分配**: 进程是资源分配的基本单位, 每个进程拥有独立内存空间; 线程是 CPU 调度的基本单位, 共享所属进程的资源。

(2) **系统开销**: 进程创建/切换需分配/回收内存等资源, 开销大; 线程切换只需保存少量寄存器内容, 开销小。

(3) **独立性**: 进程间相互独立, 一个进程崩溃不影响其他进程; 同一进程的线程共享内存, 一个线程崩溃可能导致整个进程终止。

(4) **通信方式**: 进程通信需通过进程间通信 (如管道、共享内存); 线程可直接读写进程数据段 (如全局变量)

Q2: 死锁产生的必要条件及解决方法?

(1) **必要条件**:

- **互斥**: 资源只能被一个进程独占使用。
- **请求与保持**: 进程已持有资源, 同时请求新资源。
- **不可剥夺**: 资源只能由持有进程主动释放。
- **循环等待**: 存在进程-资源的环形等待链

(2) **解决方法**:

- **预防**: 破坏任一必要条件 (如资源一次性分配、允许资源剥夺)。
- **避免**: 通过银行家算法动态检测资源分配是否安全。
- **检测与恢复**: 周期性检测死锁并终止进程或回滚资源。

Q3: 常见进程调度算法有哪些?

(1) **FCFS (先来先服务)**: 按到达顺序调度, 简单但长作业易导致“饥饿”。

(2) **SJF (短作业优先)**: 选择预计执行时间最短的作业, 需预知时间且长作业可能“饥饿”

(3) **优先级调度**: 分抢占式和非抢占式, 高优先级任务优先执行, 需解决优先级反转问题

(4) **时间片轮转 (RR)**: 每个进程分配固定时间片, 适用于分时系统。

(5) **多级反馈队列**: 结合优先级和时间片, 进程在不同队列间移动, 兼顾响应时间和吞吐量。

Q4: 进程间通信方式有哪些?

(1) **共享内存**: 通过映射同一物理内存区域实现高效通信, 需同步机制 (如信号量)。

(2) **管道 (Pipe)**: 半双工通信, 适用于父子进程, 容量有限且需同步读写。

(3) **消息队列**: 通过内核维护的消息链表传递结构化数据, 支持异步通信。

(4) **信号量 (Semaphore)**: 用于进程同步, 控制对临界资源的访问。

(5) **套接字 (Socket)**：支持网络通信，适用于分布式系统。

Q5：虚拟内存的作用及实现方式？

(1) **作用**：扩展物理内存容量，允许进程部分装入内存运行，通过置换算法（如 LRU）管理页面。

(2) **实现方式**：

- **分页**：逻辑地址划分为固定大小页，物理内存分块（页框），通过页表映射。
- **分段**：按逻辑模块划分段，每段独立地址空间，支持动态扩展和保护。
- **区别**：分页基于物理需求，分段基于逻辑需求；分页无外部碎片，分段有外部碎片。

Q6：系统调用与中断的区别？

(1) **中断**：由硬件或软件触发（如 I/O 完成），CPU 暂停当前任务处理中断，分硬中断（硬件触发）和软中断（指令触发）。

(2) **系统调用**：用户态程序通过软中断（如 `int 0x80`）主动请求内核服务（如文件读写），需切换至内核态执行。

Q7：线程共享哪些资源？

线程共享进程的堆、全局变量、静态变量、文件描述符；独立拥有栈、寄存器、程序计数器。

Q8：什么是虚拟内存？什么是共享内存？

虚拟内存：使用硬盘空间模拟内存，使进程可使用比实际物理内存更大的地址空间，提高内存利用率和系统稳定性。

共享内存：多个进程共享同一块内存区域，用于进程间通信，速度快，效率高，但需同步机制防止数据竞争。

Q9：说一说操作系统中缓冲区溢出怎么处理？

缓冲区溢出是指当数据写入缓冲区时，超过了缓冲区的容量，导致数据溢出到相邻的内存区域。这种情况可能会导致程序崩溃、数据损坏或甚至被利用来执行恶意代码。缓冲区溢出的类型包括**上溢**和**下溢**。

上溢是指超长的数据进入到缓冲区时，超出部分被写入上级缓冲区，上级缓冲区存放的可能是数据、上一条指令的指针，或者是其他程序的输出内容，这些内容都被覆盖或者破坏掉。

下溢是指超长的数据进入到缓冲区时，超出部分被写入下级缓冲区，下级缓冲区存放的是下一条指令的指针，或者是其他程序的输出内容。

防止缓冲区溢出的措施：

- **GS 保护**: 通过在函数前后添加额外的处理代码,检测缓冲区溢出并终止进程。
- **SafeSeh 保护**: 通过在 PE文件中加入异常处理例程的地址表,防止SEH 节点被攻击者恶意利用。
- **ASLR(地址空间布局随机化)保护**: 通过对系统关键地址的随机化,防止攻击者利用固定的地址定位到恶意代码。
- **DEP 保护(数据执行保护)**: 通过在内存上执行额外检查,防止在不可运行的内存区域上执行代码。

Q10: 磁盘调度算法以及磁盘空间存储管理?

磁盘调度算法: 磁盘调度算法是指操作系统用来管理磁盘 I/O 请求的顺序和时机的算法。

- **先到先服务算法(FCFS)**: 按照请求到达的顺序进行服务。
- **最短寻道时间优先算法(SSTF)**: 优先处理寻道距离最短的请求。减少了平均寻道时间,但可能导致饥饿现象。
- **扫描算法(SCAN)**: 磁盘头从一端移动到另一端,依次处理所有请求。避免了饥饿现象,但对各个位置磁道的响应频率不平均。
- **循环扫描算法(C-SCAN)**: 类似 SCAN,但每次扫描完成后,磁盘头回到起始端,返回途中不响应任何请求。

磁盘空间存储管理: 磁盘空间存储管理是指操作系统用来管理磁盘空间的算法和数据结构。

- **连续分配**: 将文件存储在连续的磁盘块中。
- **链式分配**: 将文件存储在非连续的磁盘块中,并使用指针连接各个块。
- **索引分配**: 将文件存储在非连续的磁盘块中,并使用索引表记录各个块的位置。
- **位图**: 使用位图来记录磁盘空间的使用情况。

二、计算机网络

Q1: OSI 模型, TCP/IP 模型都有哪几层, 每层的功能是什么?

(1) **OSI 模型**: 开放式系统互联通信参考模型, open system interconnection reference model, 物理层, 数据链路层, 网络层, 传输层, 会话层, 表示层, 应用层。

物理层: 负责通过物理介质(如电缆、光纤)传输原始比特流, 定义电压、光信号等物理连接特性。

数据链路层: 通过 MAC 地址寻址和帧封装实现相邻节点间的可靠数据传输, 并处理差错控制与流量管理。

网络层: 使用 IP 地址进行逻辑寻址和路由选择, 确保数据包跨网络传输。

传输层: 通过 TCP/UDP 协议提供端到端通信, 实现可靠传输或高效传输。

会话层: 负责建立、维护和终止应用程序间的会话连接(如 SSH 远程登录)。

表示层: 完成数据格式转换(如加密解密、字符编码)以实现跨系统兼容性。

应用层: 直接为用户提供 HTTP、FTP 等网络服务接口, 实现文件传输、邮件收发等功能。

(2) **TCP/IP 模型**: 网络接口层, 网络层, 传输层, 应用层。

网络接口层: 对应 OSI 的物理层和数据链路层, 负责将 IP 数据封装为物理网络可识别的帧格式(如以太网帧), 并通过 MAC 地址完成物理介质上的数据传输。

网络层(或称网际互联层): 通过 IP 协议实现数据包的逻辑寻址、路由选择和跨网络转发, 并包含 ARP、ICMP 等辅助协议。

传输层: 通过 TCP 或 UDP 协议提供端到端通信服务, 确保数据可靠传输(如 TCP 三次握手)或高效无连接传输(如 UDP 实时流媒体)。

应用层: 负责整合 OSI 模型的应用层、表示层和会话层功能, 直接为用户提供 HTTP、FTP、SMTP 等网络服务接口以实现文件传输或网页访问;

Q2: HTTP 与 HTTPS 的核心区别?

(1) **安全性**: HTTP 明文传输, 易被窃听/篡改; HTTPS 通过 SSL/TLS 加密, 保障数据机密性与完整性。

(2) **端口**: HTTP 默认 80 端口, HTTPS 默认 443 端口。

(3) **证书**: HTTPS 需 CA 颁发的数字证书验证服务器身份。

(4) **性能**: HTTPS 因加密握手增加延迟, 但 HTTP/2 后差距缩小。

Q3: TCP 三次握手与四次挥手的流程及设计原因?

(1) **第一次握手(SYN)**: 客户端发送 SYN 报文(SYN=1), 携带随机生成的初始序列号 seq=x, 并进入 SYN_SENT 状态。

(2) **第二次握手(SYN+ACK)**: 服务器收到 SYN 后, 返回 SYN+ACK 报文(SYN=1, ACK=1), 携带自身初始序列号 seq=y 及确认号 ack=x+1, 进入 SYN_RCVD 状态。

(3) **第三次握手(ACK)**: 客户端发送 ACK 报文(ACK=1), 携带 ack=y+1, ESTABLISHED 状态。

原因: 确保双方收发能力正常, 避免历史连接干扰。

- (1) **第一次挥手(FIN)**: 主动关闭方(如客户端)发送 FIN 报文(FIN=1, seq=u), 进入 FIN_WAIT_1 状态。
- (2) **第二次挥手(ACK)**: 被动关闭方(如服务器)返回 ACK 报文(ACK=1, ack=u+1), 进入 CLOSE_WAIT 状态, 客户端收到后进入 FIN_WAIT_2。
- (3) **第三次挥手(FIN)**: 服务器发送 FIN 报文(FIN=1, seq=w), 进入 LAST_ACK 状态。
- (4) **第四次挥手(ACK)**: 客户端发送 ACK 报文(ACK=1, ack=w+1), 进入 TIME_WAIT 状态, 等待 2MSL (最大报文生存时间) 后关闭; 服务器收到 ACK 后立即关闭。
- 原因**: TCP 全双工特性, 需双方独立关闭通道。

Q4: TCP 如何保证可靠传输?

- (1) **校验和**: 检测数据包是否损坏。
- (2) **确认应答与超时重传**: 接收方返回 ACK, 发送方未收到则重传。
- (3) **流量控制**: 滑动窗口机制动态调整发送速率。
- (4) **拥塞控制**: 慢启动、拥塞避免、快速重传/恢复算法 (如 Reno)。

Q5: 在 TCP 拥塞控制中, 什么是慢开始、拥塞避免、快重传和快恢复算法?

慢开始 (Slow Start):

- **目的**: 探测网络可用带宽, 逐步增加发送窗口大小。
- **过程**: 初始拥塞窗口 (cwnd) 设为 1, 随后每收到一个 ACK, cwnd 加倍, 指数增长, 直到达到慢开始门限 (ssthresh)。

拥塞避免 (Congestion Avoidance):

- **目的**: 避免网络拥塞, 平稳增长窗口大小。
- **过程**: 当 cwnd 达到 ssthresh 后, 每次收到 ACK, cwnd 增加 1 (线性增长)。

快重传 (Fast Retransmit):

- **目的**: 尽早发现丢包, 迅速重传丢失的数据段。
- **过程**: 当发送方连续收到 3 个重复 ACK 时, 立即重传被确认丢失的数据段, 而不必等待重传计时器到期。

快恢复 (Fast Recovery):

- **目的**: 快速恢复传输效率, 避免进入慢开始阶段。
- **过程**: 在快重传重传丢失的数据段后, 将 ssthresh 设置为 cwnd 的一半, 然后 cwnd 设为 ssthresh, 进入拥塞避免阶段, 继续线性增长。

Q6: 流量控制和拥塞是什么关系?

流量控制 (Flow Control):

- **目的**: 防止发送方发送数据过快, 接收方处理不过来, 导致丢包。
- **机制**: 通过滑动窗口协议, 接收方告知发送方自己能接收的窗口大小, 发送方根据这个窗口大小调节发送速率。

拥塞控制 (Congestion Control) :

- **目的:** 防止网络出现拥塞, 确保网络稳定运行。
- **机制:** 包括慢开始、拥塞避免、快重传和快恢复等算法, 通过调整发送窗口大小来探测和避免网络拥塞。

关系: 流量控制针对的是发送方和接收方之间的速率匹配, 确保接收方不被淹没; 拥塞控制则针对整个网络环境, 防止网络整体过载, 保证网络资源的合理使用和数据的有效传输。

Q7: TCP 和 UDP 的异同点

TCP (传输控制协议) :

- **连接导向:** 需要建立连接 (通过三次握手)。
- **可靠传输:** 有确认机制、重传机制和流量控制, 确保数据不丢失、按序到达。
- **头部较大:** 因为有更多的控制信息 (20 字节)。
- **应用场景:** 适用于对可靠性要求高的应用, 如 HTTP、FTP、SMTP。

UDP (用户数据报协议) :

- **无连接:** 不需要建立连接, 直接发送数据。
- **不可靠传输:** 没有确认和重传机制, 可能丢包, 数据可能乱序。
- **头部较小:** 仅 8 字节, 效率较高。
- **应用场景:** 适用于对实时性要求高的应用, 如 DNS 查询、视频直播、在线游戏。

Q8: ARP 协议的作用与工作原理?

(1) **作用:** 将 IP 地址解析为 MAC 地址。

(2) **流程:** 主机广播 ARP 请求包, 目标 IP 主机单播回复 MAC 地址。

(1) 首先, 每个主机都会在自己的 ARP 缓冲区中建立一个 ARP 列表, 以表示 IP 地址和 MAC 地址之间的对应关系。

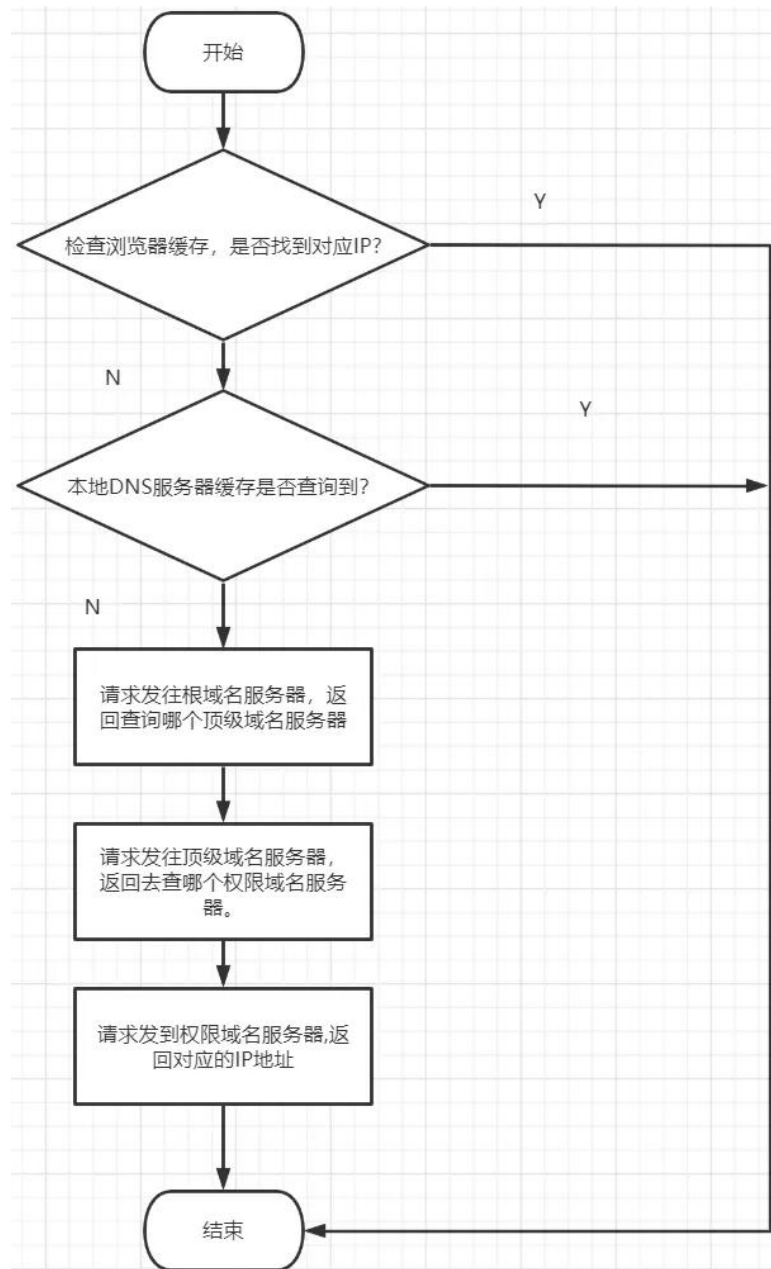
(2) 当源主机要发送数据时, 首先检查 ARP 列表中是否有对应 IP 地址的目的主机的 MAC 地址, 如果有, 则直接发送数据, 如果没有, 就向本网段的所有主机发送 ARP 数据包, 该数据包包括的内容有: 源主机 IP 地址, 源主机 MAC 地址, 目的主机的 IP 地址。

(3) 当本网络的所有主机收到该 ARP 数据包时, 首先检查数据包中的 IP 地址是否是自己的 IP 地址, 如果不是, 则忽略该数据包, 如果是, 则首先从数据包中取出源主机的 IP 和 MAC 地址写入到 ARP 列表中, 如果已经存在, 则覆盖, 然后将自己的 MAC 地址写入 ARP 响应包中, 告诉源主机自己是它要找的 MAC 地址。

(4) 源主机收到 ARP 响应包后。将目的主机的 IP 和 MAC 地址写入 ARP 列表, 并利用此信息发送数据。如果源主机一直没有收到 ARP 响应数据包, 表示 ARP 查询失败。

Q9: DNS 解析过程?

浏览器缓存 → 2. 本地 Hosts 文件 → 3. 本地 DNS 服务器 → 4. 根域名服务器 → 5. 顶级域名服务器 (如 .com) → 6. 权威域名服务器 (返回目标 IP)。



Q10: DHCP 的作用?

DHCP (动态主机配置协议) 的主要作用是自动分配网络参数, 简化网络管理。具体功能包括:

- **分配 IP 地址:** 自动分配 IP 地址给连接到网络的设备。
- **配置网络参数:** 提供子网掩码、网关地址和 DNS 服务器等网络配置。
- **地址租赁:** 管理 IP 地址租期, 确保地址不会冲突, 并允许设备在租期到期后续租或释放地址。

三、数据结构与算法

Q1: 线性存储结构和链式存储结构的优点

(1) 线性存储结构地址空间连接，可随机访问，但是顺序存储的缺点是删除，插入操作需要花费很多时间在移动元素上。

(2) 链式存储结构不要求逻辑上相邻的元素在物理位置上是相邻，但也同时失去了顺序表可随机存取的优点，但方便插入与删除。

Q2: 栈和队列定义及其存储结构

(1) **栈的定义**是只允许在表的末端进行插入和删除的线性表，允许插入和删除的一端叫做栈顶，不允许插入和删除的一端叫做栈底。**栈的存储结构**通常分为基于数组的存储表示和顺序存储结构和基于链表的链式存储结构。**顺序栈的内存结构**：存放栈中元素的数组、栈顶指针、最大容纳元素个数。**链式栈的内存结构**：栈顶指针

(2) **队列的定义**是只允许在表的一端插入，在另一端删除的线性表。允许插入元素的一端称为队尾，允许删除的一端称为队首。**队列的存储结构**通常分为基于数组的存储表示和基于链表的存储表示。**顺序存储结构**利用一个一维数组作为存储结构，并设置头指针和尾指针两个指针来指示队首和队尾的位置。**链式队列**由队首指针和队尾指针构成。

Q3: 判断链表是否有环

快慢指针：从头开始设置两个指针，快指针每次走 2 步，慢指针每次走 1 步，如果快指针先碰到尾，则无环，否则两个指针之后一定会重合，则有环。

Q4: 最小生成树和最短路径用什么算法来实现？（迪杰斯特拉、弗洛伊德、普利姆、克鲁斯卡尔）算法的基本思想是什么？算法的时间复杂度？

(1) **最小生成树**：普利姆、克鲁斯卡尔

(2) **最短路**：迪杰斯特拉、弗洛伊德，单源负权边时是 Bellman Ford 算法

(3) **算法基本思想**：迪杰斯特拉，克鲁斯卡尔，普利姆是贪心；floyd 是动态规划，也就是枚举中间点， $i-j$ 的距离可以被优化成经过中间点 k , $i-k-j$ 。

(4) **算法时间复杂度**：堆优化的迪杰斯特拉，克鲁斯卡尔，普利姆均为 $O(n \log n)$ ，普通 dji 是 $O(n^2)$ ，floyd 为 $O(n^3)$

Q5: 插入排序、希尔排序、选择排序、冒泡排序、归并排序、快速排序、堆排序、基数排序等排算法的基本思想是什么？时间复杂度？是

否稳定？

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n\log_2 n)$	不稳定
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

插入排序：将数组划分为已排序和未排序部分，依次将未排序部分的元素插入到已排序部分的适当位置。

希尔排序：对间隔为 h 的元素进行插入排序，并逐渐减少 h 的值直到 $h=1$ 。

选择排序：在未排序部分中选择最小元素并交换到已排序部分的末尾。

冒泡排序：反复交换相邻的逆序元素，使每次遍历后最大元素移动到末尾。

归并排序：将数组递归地分成两半，对每一半分别排序后再合并。

快速排序：选择一个基准元素，将数组分为小于和大于基准元素的两部分，再递归排序这两部分。

堆排序：构建最大堆，每次取出堆顶元素并调整堆。

基数排序：按位排序，从最低位开始依次进行稳定排序。

Q6：讲解一下邻接表和邻接矩阵（又如何存储大数据？）

邻接表（Adjacency List）和邻接矩阵（Adjacency Matrix）是存储图的两种常见方法。

邻接表：

- 每个顶点存储一个链表，链表中的每个节点表示一个与该顶点相邻的顶点。
- 适用于稀疏图，节省空间。
- 存储大数据时，可以使用链表或哈希表等动态数据结构。

邻接矩阵：

- 使用一个二维数组表示顶点之间的连接关系，数组元素为 1 表示有边，为 0 表示无边。
- 适用于稠密图，但空间复杂度高（ $O(n^2)$ ）。

- 存储大数据时，可以使用稀疏矩阵的存储方式，如压缩存储。

Q7：介绍一下深度优先搜索和广度优先搜索是如何实现的？

深度优先搜索（DFS）：

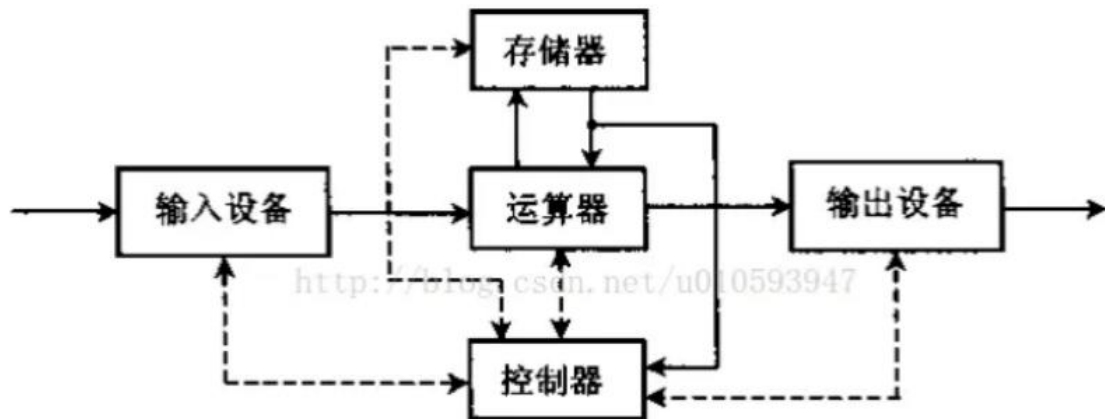
- **实现：**使用递归或栈，从起始节点开始，深入访问每一个未访问的节点，直到不能继续为止，然后回溯。
- **应用：**拓扑排序、连通分量、检测环。

广度优先搜索（BFS）：

- **实现：**使用队列，从起始节点开始，访问所有相邻节点，然后依次访问这些节点的相邻节点。
- **应用：**最短路径（无权图）、层次遍历。

四、计算机组成原理

Q1：冯诺依曼机的体系结构



主要由五大部件组成：

存储器用来存放数据和程序。

运算器主要运行算术运算和逻辑运算，并将中间结果暂存到运算器中。

控制器主要用来控制和指挥程序和数据的输入运行，以及处理运算结果。

输入设备用来将人们熟悉的信息形式转换为机器能够识别的信息形式，常见的有键盘，鼠标等。

输出设备可以将机器运算结果转换为人们熟悉的信息形式，如打印机输出，显示器输出等。

Q2：衡量计算机性能指标

吞吐量：表征一台计算机在某一时间间隔内能够处理的信息量，单位是字节/秒。

响应时间：表征从输入有效到系统产生响应之间的时间度量。

利用率：表示在给定的时间间隔内，系统被实际使用的时间所占的比率。

处理机字长：指处理机运算器中一次能够完成二进制数运算的位数，字长越长，表示计算的精度越高。

总线宽度：一般指 CPU 中运算器与存储器之间进行互连的内部总线二进制位数。

存储器容量：存储器中所有存储单元的总数目，通常用KB、MB、GB、TB 来表示。

存储器带宽：存储器的速度指标，单位时间内从存储器读出的二进制数信息量，一般用字节数/秒表示。

主频/时钟周期：CPU 的工作节拍受主时钟控制，主时钟不断产生固定频率的时钟，主时钟的频率（f）叫 CPU 的主频。度量单位是 MHz、GHz。主频的倒数称为 CPU 时钟周期。

CPU 执行时间：表示 CPU 执行一段程序所占用的 CPU 时间， $\text{CPU 执行时间} = \text{CPU 时钟周期数} \times \text{CPU 时钟}$ 。

周期长 CPI：表示每条指令周期数，即执行一条指令所需的平均时钟周期数。

Q3: CPU一个指令周期的流程是什么？

取指 (Fetch)： 从内存中读取指令到指令寄存器。

译码 (Decode)： 分析指令，确定操作码和操作数。

执行 (Execute)： 根据指令执行相应操作，如算术运算或数据传输。

访存 (Memory Access)： 若指令需要，访问内存读取或写入数据。

写回 (Write Back)： 将结果写回寄存器或内存。

Q4: 存储器的分类 (RAM、ROM 的区别)

存储器是计算机系统中用于存储数据和程序的设备。主要分为 RAM 和 ROM。

RAM (随机存取存储器)：

- **用途：** 作为 CPU 的直接数据交换媒介，存储操作系统、应用程序和当前运行的数据。
- **特性：** 断电后数据会丢失，可以随时读写，通常比长期存储设备快。

RAM又分为 SRAM 和 DRAM。**SRAM (静态随机存取存储器)** 不需要刷新电路，数据保持稳定。速度快，适合关键性系统。但成本高，集成度低，功耗大。**DRAM (动态随机存取存储器)** 使用电容存储数据，需要定期刷新。成本较低，集成度高。但需要刷新电路，数据易丢失。

ROM (只读存储器)：

- **用途：** 存储固定数据和程序，如 BIOS、固件。
- **特性：** 断电后数据不丢失。主要用于读取，写入通常在制造时完成。

Q5: 存储系统的层次结构

现代计算机通常采用三级存储结构：

高速缓存 (Cache)： 速度最快，容量最小，存放 CPU 频繁访问的数据副本。

主存储器 (内存)： 作为核心工作区，临时存储运行中的程序。

辅助存储器 (外存)： 提供海量存储空间，如硬盘和云存储。

Q6: 总线通讯的四种方式

同步通讯： 发送和接收双方使用同一时钟信号同步数据传输，适用于高频短距离通信。

异步通讯： 发送和接收双方独立工作，通过起始和停止位同步，适用于低频长距离通信。

半双工通讯： 同一时刻只能单方向传输数据，发送和接收不能同时进行。

全双工通讯： 可以同时进行双向数据传输，提高了通信效率。

Q7: 中断处理流程

中断请求→中断判优→保存现场→执行服务程序→恢复现场→返回

Q8: 寻址方式

立即寻址: 操作数直接编码在指令中, 无需访问内存或寄存器。

直接寻址: 指令中的地址码字段直接给出操作数的有效地址 (EA)。

寄存器间接寻址: 寄存器中存储的是操作数的地址, 需通过寄存器间接访问内存。

基址寻址: 有效地址 = 基址寄存器内容 + 偏移量 (由指令给出)

变址寻址: 有效地址 = 变址寄存器内容 + 基地址 (指令中给出)

组合寻址 (基址变址寻址): 有效地址 = 基址寄存器 + 变址寄存器 + 偏移量